Java OOP Project Phase 1 Instructions

For this project, you will choose a Java application that interests you and develop it in two phases. The goal is to practice the object-oriented programming concepts covered in class by building a working application, with core classes that you can reuse when you add a graphical interface.

Phase 1 focuses on the core application. Build a console-based application, or implement the core logic and demonstrate it through a runnable console driver. Your project must use multiple classes, inheritance, and packages and contain at least 500 lines of Java application code.

In Phase 2, you will continue working on the same project by adding a graphical user interface, event handling, and additional features.

1 Choose your project

Choose one of the project ideas below, or propose a different idea and obtain my approval before you begin. The suggested features and class names are starting points that you can adapt to your project. Regardless of the topic you choose, your work must meet the common requirements in Sections 2 through 4.

A detailed console chess example appears at the end of this document. You may use it as a reference for the expected scope and design, even if you choose a different project.

Project idea	Possible Phase 1 features	Example inheritance and behavior
Console chess	Display a board, validate moves, alternate turns, and track captures.	Piece → Pawn, Rook, Knight, Bishop, Queen, King; different movement rules.
Library lending system	Search a catalog, manage members, check out and return items, and track loans.	LibraryItem → Book, DVD; different loan periods or late-fee rules.
Task and study planner	Manage tasks, priorities, deadlines, completion, and recurring work.	Task → OneTimeTask, RecurringTask; different next-due-date behavior.
Store inventory and checkout	Manage stock, build orders, validate quantities, and generate receipts.	Product → PhysicalProduct, DigitalProduct; different delivery or fulfillment behavior.
Pet adoption manager	Manage animal profiles, search, record applications, and update adoption status.	Animal → Dog, Cat; different care or housing requirements.
Turn-based adventure game	Explore locations, manage inventory, and resolve battles with special abilities.	GameCharacter → Warrior, Mage; different attacks and resource use.

Project idea	Possible Phase 1 features	Example inheritance and behavior
Vehicle rental system	Check vehicle availability, make reservations, process returns, and calculate charges.	Vehicle → Car, Motorcycle; different rental-price calculations.
Quiz and practice app	Build question banks, run quizzes, grade responses, and review results by topic.	Question → MultipleChoiceQuestion, ShortAnswerQuestion; different validation and grading.
Restaurant ordering system	Manage menu items, customize orders, calculate totals, and track order status.	MenuItem → FoodItem, DrinkItem; different customization and price rules.
Fitness workout planner	Build workouts, record completed activities, and summarize training sessions.	Exercise → StrengthExercise, CardioExercise; different performance metrics.
Event ticket booking system	List events, manage seating or capacity, book and cancel tickets, and show bookings.	Ticket → StandardTicket, VIPTicket; different prices and included benefits.
Bank account simulator	Manage fictional accounts, deposits, withdrawals, transfers, and transaction history.	Account → CheckingAccount, SavingsAccount; different withdrawal or interest rules.

If you propose another idea

Prepare a short description of what your application will do, who will use it, and which features you plan to complete in Phase 1. Discuss your proposal with me and obtain approval before you begin implementing it.

Choose a scope you can complete

- Choose features that give your classes meaningful responsibilities and require objects to work together. Your subclasses should differ in behavior, so that inheritance serves a clear purpose in the application.
- Plan enough functionality to meet the 500-line minimum through useful features. A basic calculator or tic-tac-toe game will need substantial extensions to meet this requirement.
- Keep the tools and setup manageable. You do not need a database, network service, or external API for Phase 1; in-memory collections and local sample data are sufficient.

2 Complete the Phase 1 requirements

Every project must meet the requirements in this section. Use the checklist to guide your development and review your work before submitting it.

Build a working application

- ☐ **Working core features.** Implement the rules, calculations, data operations, or game mechanics needed for your main features to work. Class definitions and empty methods alone do not meet this requirement.
- ☐ **Runnable demonstration.** Provide either a console interface that allows repeated operations and exits cleanly, or a console driver that runs complete scenarios and clearly labels the inputs and results.
- ☐ **Input and operation validation.** Handle invalid input and invalid operations with clear messages. If an operation fails, the application must preserve its valid state.

Demonstrate object oriented design

- ☐ **Multiple classes.** Give each class a clear responsibility, and place each public class in its own appropriately named .java file. Keep main focused on starting the application, with the application logic handled by other classes.
- ☐ **Encapsulation.** Use appropriate access modifiers, constructors, and methods to protect your data and keep objects in a valid state.
- ☐ **Inheritance.** Include at least one meaningful superclass and multiple subclasses with a valid “is a” relationship. Place shared data or behavior in the superclass so that subclasses can reuse it.
- ☐ **Polymorphism.** Override behavior in your subclasses and demonstrate calls to those methods through superclass references, either individually or in a collection. Use an abstract class or interface where it fits your design.
- ☐ **Packages and object collaboration.** Organize your classes into at least two named packages based on their responsibilities. Keep the core application logic separate from console input and output so that the same logic can support the GUI in Phase 2.

Meet the code and development requirements

- ☐ **Code minimum.** Include at least 500 lines of Java application code. Count only nonblank, non-comment lines in your application’s .java files.
- ☐ **Readable code and Javadoc.** Use consistent formatting and descriptive names. Write Javadoc for classes, interfaces, methods, and attributes, including parameter and return information where relevant, and generate the HTML documentation.
- ☐ **GitHub history.** Make multiple meaningful commits as you develop the project, using messages that explain what changed. Verify that I can access your repository and its commit history.

3 Follow the AI use policy

You may use AI-assisted coding and vibe coding for this project. This includes using AI to brainstorm, generate, refactor, debug, test, or document project code. You remain fully responsible for everything you submit, including code and documentation produced with AI assistance.

How you may use AI

- Use AI to explore project ideas, plan features, discuss class responsibilities, or better understand the OOP concepts used in your design.
- You may ask AI to generate classes, methods, or other project code, and you may use AI code completion or a conversational coding workflow. You may also use AI to refactor existing code and suggest or implement fixes.
- AI may help you develop test cases, investigate errors, and prepare documentation. Review its suggestions and check them against the behavior of your actual application before including them in your submission.

Your responsibility for the submitted work

- Read and understand the code you submit, including any code generated by AI. You should be able to explain how the main features work, how objects interact, and why the design meets the project's OOP requirements.
- Run and test the application yourself. Check normal operations, invalid input, and relevant boundary cases, and correct problems you find. An AI tool's claim that the code works is not a substitute for verifying it.
- Make sure the final application meets every Phase 1 requirement. Check that the README, UML diagram, Javadoc, and report describe the version you are submitting, and disclose any known limitations.

I may ask you to explain a line of code, trace an operation, justify a design choice, or make a small change. Using AI is permitted and does not by itself reduce your grade. Your work will be evaluated using the rubric in Section 5, including your understanding of the submitted code.

Required AI disclosure in the report

If you did not use AI, write: "I did not use AI tools for this project."

If you used AI, identify the tools and explain how you used them. Include representative prompts or a brief interaction summary, identify the main parts of the project that AI helped generate or revise, and describe how you reviewed, tested, and corrected the output.

End your disclosure with this statement: "I have reviewed and tested the work I am submitting. I understand the code and take responsibility for its correctness, documentation, and compliance with the project requirements." Include this statement whether or not you used AI.

4 Submit your Phase 1 work

Submit the following items through the course assignment page by the posted deadline. Follow any file naming instructions provided for the assignment.

Submission checklist

- ☐ **GitHub repository link.** Include the link in your report and confirm that I can access both the project files and the commit history.
- ☐ **README.** Explain the application's purpose, required Java version, entry-point class, and setup and run instructions. List completed features and known limitations, and provide sample input or instructions for running your demonstration driver.
- ☐ **Generated Javadoc.** Include the generated HTML documentation, along with its index.html entry point.
- ☐ **Project report.** Submit a PDF or Word document containing the four sections listed below.

Required report sections

1. Project summary. Include the project title, student name or names, the application's purpose and intended users, and the features you completed in Phase 1.

2. Object-oriented design. Include a UML class diagram that shows the main classes, inheritance relationships, important associations, and package organization.

3. Demonstration and testing. Include screenshots of console output showing the main features. Also show how you checked invalid input and relevant boundary cases, with a brief explanation of the expected and observed results.

4. AI disclosure. Provide the disclosure described in Section 3, whether or not you used AI.

5 Grading rubric

Phase 1 is worth 100 points, and it worth 15% of your final grade. The same rubric applies to every project, with functionality judged against the features and scope of the chosen application. The examples below show typical scores; intermediate scores may be awarded when the work falls between them.

Grading area	Points
Application functionality and validation	30
Object oriented design	25
Code organization and project size	10
Demonstration and testing	10
README and Javadoc	10
GitHub development history and access	5
Project report and UML diagram	5
AI disclosure and understanding of the code	5
Total	**100**

Application functionality and validation 30 points

Core features are worth 18 points, the console interface or demonstration driver 6 points, and validation 6 points. Full credit means the main workflows run correctly and invalid operations produce useful messages without damaging application state. The console interface should support repeated operations and a clean exit, or the driver should run complete scenarios with labeled inputs and results.

For example, award 30 points when all three areas work well; 24 points when the core features and console work but validation is missing; or 15 points when about half the core functionality works (9), the console is usable (6), and validation is absent (0). Award 0 when there is no working application behavior to assess.

Object oriented design 25 points

Award up to 8 points for a meaningful superclass with multiple subclasses, 7 for overridden behavior called through superclass references, 6 for encapsulation, and 4 for clear class responsibilities and object collaboration.

A design that meets all four expectations earns 25 points. A project with appropriate inheritance, encapsulation, and collaborating classes, but no demonstrated polymorphic calls, earns 18. Merely declaring a superclass and subclasses with no useful shared or specialized behavior earns at most 2 of the 8 inheritance points.

Code organization and project size 10 points

Award 3 points for at least two packages organized by responsibility, 3 for separating core logic from console input and output, 2 for readable formatting and names with appropriately named public-class files, and 2 for at least 500 nonblank, non-comment lines of Java application code.

A project meeting all four expectations earns 10 points. If everything else is satisfactory but the project has fewer than 500 qualifying lines, award 8. If it also uses only one package, award 5. Give partial credit for incomplete separation or inconsistent readability; empty packages and repetitive filler do not satisfy the intended requirements.

Grading rubric continued

Demonstration and testing 10 points

Award up to 6 points for clear console evidence covering the main features and 4 for checks of invalid input and relevant boundary cases, with expected and observed results. Manual console tests are sufficient; an automated test framework is not required.

A complete demonstration with both kinds of checks earns 10 points. Screenshots covering the main features but only successful inputs earn 6. A single screenshot showing startup, with no completed operation or test results, earns 1; no demonstration or test evidence earns 0.

README and Javadoc 10 points

Award up to 4 points for a complete, accurate README, 4 for useful Javadoc covering the required code elements, and 2 for generated HTML documentation with a working index.html entry point.

Complete documentation earns 10 points. A complete README and source Javadoc without generated HTML earn 8. A README with reliable run instructions but missing features and limitations earns 2 of its 4 points. Missing or unusable items earn 0 for the corresponding part.

GitHub development history and access 5 points

Award 3 points for multiple meaningful commits that show development over time and 2 for an accessible repository containing the submitted project and history. A complete history with useful messages earns 5 points. An accessible repository with one final upload earns 2; multiple commits with vague messages and little visible progression may earn 3 or 4 overall. Award no access points if I cannot open the repository.

Project report and UML diagram 5 points

Award up to 2 points for a clear project summary with the required identifying information and completed features, and 3 for an accurate UML diagram showing the main classes, inheritance, important associations, and packages. A complete summary and diagram earn 5 points. A complete summary with a diagram missing associations and package organization earns 3; the summary alone earns 2. Screenshots and AI disclosure are scored in their own categories.

AI disclosure and understanding of the code 5 points

Award up to 2 points for the disclosure required in Section 3 and 3 for demonstrated understanding of the submitted code. Full credit applies equally to students who use AI and those who do not. A complete disclosure and clear explanation of the design and behavior earn 5 points. A complete disclosure with only a limited explanation of how the code works may earn 3; a missing disclosure with a clear explanation earns 3. Award 0 for understanding if the student cannot explain the submitted logic.

Each category is scored separately. A problem that prevents the application from running affects functionality, but any submitted design, documentation, or other evidence can still earn credit in its own category. Optional features do not replace the required work or raise the total above 100 points.

6 Example project Console chess

If you choose console chess, build a two-player application using the board, piece hierarchy, and turn-based workflow described in this example. All projects must follow Sections 1 through 4 and will be graded using Section 5. The chess-specific requirements below apply only to the chess option.

For the chess option, you must implement the board, all six piece types, basic movement and capture rules, input validation, and alternating turns. You may leave out the advanced rules identified at the end of this example for your Phase 1 prototype.

Board display and piece symbols

Display an 8 × 8 board with files A–H across the top and ranks 8–1 down the left. White begins on ranks 1–2, and Black begins on ranks 7–8. Use ## for an empty square. For each piece, use w or b to indicate its color, followed by p, R, N, B, Q, or K for pawn, rook, knight, bishop, queen, or king.

	A	B	C	D	E	F	G	H
8	bR	bN	bB	bQ	bK	bB	bN	bR
7	bp	bp	bp	bp	bp	bp	bp	bp
6	##	##	##	##	##	##	##	##
5	##	##	##	##	##	##	##	##
4	##	##	##	##	##	##	##	##
3	##	##	##	##	##	##	##	##
2	wp	wp	wp	wp	wp	wp	wp	wp
1	wR	wN	wB	wQ	wK	wB	wN	wR

The table above shows the starting position. Your application should display the same information as text in the console.

Move input and console workflow

Accept moves in the form FROM TO. For example, E2 E4 moves the piece on E2 to E4. Use the same format for captures; the program should detect whether the destination contains an opponent's piece. White moves first, and the application must provide a clear quit command.

1. Display the board and identify whose turn it is.
2. Prompt the player to enter a move or the quit command. For a move, parse the source and destination coordinates.
3. Validate the input format and coordinates, then check piece ownership, the destination square, the piece's movement pattern, and whether its path is blocked.
4. If the move is valid, update the board and captured-piece record, then switch turns. If the move is invalid, explain the problem and leave both the board and the current turn unchanged.
5. Display the updated board and continue the loop. Exit cleanly when the player enters the quit command. Announce a winner or draw only if you have implemented the rule needed to determine that outcome.

Chess example Class design

The classes and method signatures below are suggestions. You may adapt them to your design as long as your application meets the required chess behavior. Put each public class in its own file.

Package	Classes
chess.pieces	Piece, Pawn, Rook, Knight, Bishop, Queen, King
chess.model	Board, Position, Player, PieceColor
chess.service	Game
chess.console	ChessApp, ConsoleUI, InputParser

Piece and its six subclasses

Use an abstract Piece class to hold the color and position data shared by all pieces. Create Pawn, Rook, Knight, Bishop, Queen, and King as subclasses.

Each subclass can override possibleMoves(Board board) to return its candidate destinations. After a move is validated, move(Position newPosition) can update the piece's position. Storing the pieces as Piece references allows the program to call the appropriate subclass implementation through polymorphism. If you omit king-safety checks, these candidate moves may not be fully legal under standard chess rules.

Board

Store the board as Piece[8][8], or use another suitable structure, with null representing an empty square. Keep a list of captured pieces as well.

Useful methods include initialize(), getPiece(Position), isPathClear(Position from, Position to), and movePiece(Position from, Position to). Validate a move before changing the board. Once it is accepted, record any capture, clear the source square, fill the destination square, and update the moving piece's position so that all stored information remains consistent.

Position and Player

Position stores a row and column and handles coordinate conversion and bounds checks. For example, if array row 0 represents rank 8, E2 maps to row 6, column 4. Document your mapping and use it consistently throughout the application.

Player stores a player's color and, optionally, a name. You can determine a player's active pieces from Board or maintain a separate list that is updated after captures. An enum named PieceColor can represent WHITE and BLACK.

Game

Game holds the Board, the two Player objects, and the current turn.

Methods such as start(), tryMove(Position from, Position to), and end() can coordinate ownership checks, move validation, board updates, and turn changes. Return a result or message that the interface can display to the player.

ConsoleUI and supporting classes

ConsoleUI displays the board, reads commands, and uses Game to run the play loop. InputParser converts the entered text into positions or a quit request, while ChessApp contains main. Add helper classes when they have a clear responsibility in your design.

To prepare for Phase 2, keep display() and any input-driven makeMove() responsibilities in the console layer. GUI event handlers can then call Game and reuse the existing board and piece rules.

Chess example Rules and testing

Required basic movement rules

Piece	Phase 1 movement behavior
Pawn	Move one square forward into an empty square; allow two from the starting rank only if both squares are empty. Capture one square diagonally forward. White advances toward rank 8; Black toward rank 1.
Rook	Move horizontally or vertically through empty intervening squares.
Bishop	Move diagonally through empty intervening squares.
Queen	Combine rook and bishop movement; intervening squares must be empty.
Knight	Move two squares on one axis and one on the other. May jump over intervening pieces.
King	Move one square in any direction. King-safety checks may be deferred as described below.

Reject moves that leave a piece on the same square, use coordinates outside the board, start from an empty square, select the wrong player's piece, or end on a friendly piece. When a capture is valid, replace the opponent's piece with the moving piece.

Suggested test scenarios

First, verify the starting arrangement and confirm that White moves first. Reset the board before each test or move sequence below. In addition to these examples, test every piece type and the boundary cases relevant to its movement.

Input or setup	Expected result
E2 E4, then E7 E5	Both accepted; positions update and White moves next.
E2 E4, D7 D5, E4 D5	White captures the black pawn; E4 becomes empty; Black moves next.
A1 A3; G1 F3	Test separately: rook is blocked; knight can jump to F3.
E2 E5; E7 E5; E2 E1; E3 E4; E2 E2	Test separately: reject invalid movement, wrong ownership, friendly destination, empty source, and unchanged position.
Z9 E4; E2; quit	Invalid input preserves state and does not crash; quit exits cleanly.

Optional advanced rules and Phase 2 extensions

Check, checkmate, stalemate, castling, en passant, and pawn promotion are optional in Phase 1. Document any rules you omit and explain what your program does when a pawn reaches the last rank. Clearly identify the application as a simplified chess prototype.

If you implement advanced rules, methods such as isCheck(color), isCheckmate(color), and isStalemate(color) may help organize the logic. Use O-O or O-O-O for castling and E7 E8=Q for a valid promotion. An implementation of the full rules must prevent moves that leave the player's own king in check.

In Phase 2, add a graphical board whose click or drag-and-drop events call Game. You can also extend the application with move history, saving and loading, a captured-piece display, or complete chess rules. Continue testing, documenting, and committing your work as you develop these features.